Instructions for *ACL Proceedings

Anonymous ACL submission

Abstract

(SP: I basically just copied Jeff's abstract with a few edits. Probably should be shortened considerably) Soubki and Heinz (2023) examine the performance of the state-merging algorithms RPNI, EDSM, and ALERGIA on an early version of MLRegTest, a benchmark for the learning of regular languages (van der Poel et al., 2024). They found that the neural models generally outperformed the state-merging algorithms on this benchmark. However, the MLRegTest benchmark excludes strings less than length 20 in order to achieve a balance of positive and negative strings in training, development and test sets across all languages in MLRegTest. Soubki and Heinz (2023) report a follow-up experiment which includes shorter strings and show that they dramatically improve the performance of the state-merging algorithms.

This paper makes three contributions. First, it replicates the experiments in Soubki and Heinz (2023) for the neural networks and state-merging algorithms RPNI and EDSM on the final version of MLRegTest. This replication includes a grid search over many parameters belonging to the state-merging algorithms provided by the FlexFringe software (Verwer and Hammerschmidt, 2017, 2022) to find suitable hyperparameters. Second, it conducts a comprehensive set of follow-experiments with shorter strings on *both* the neural and state-merging learning systems to allow for comparison. Third, on the basis of these experiments, we conclude that the neural systems outperform the state-merging systems in all conditions save one. When the training data is small, and the short strings are included, EDSM outperforms the neural networks.

1 Introduction

This paper compares the results of a state merging algorithm and four neural network architectures on an extension of MLRegTest, a benchmark for the learning of regular languages (van der Poel et al., 2024). The training data of MLRegTest contains only strings of at least length 20, but Soubki and Heinz (2023) demonstrated that augmenting this training data with shorter strings led to significant improvement in the performance of the state merging algorithms. Following Soubki and Heinz, we compare performance of our models when trained on the original MLRegTest data, the augmented Soubki and Heinz data, and training data containing *only* strings of length 19 or less. We find that (TODO: let's see what we find once everything is done :))

2 Learning Systems

2.1 State-Merging Systems

Following Soubki and Heinz (2023), we make use of the state-merging algorithms provided by FlexFringe (Verwer and Hammerschmidt, 2017, 2022).[1] FlexFringe provides implementations of the EDSM (Lang et al., 1998), RPNI (Oncina et al., 1992)state-merging algorithms:[2] both take as input a set of strings labeled as to whether or not they belong to the target language, and construct a prefix tree from these strings. A search is then conducted for pairs of states to merge: RPNI does so in a breadth-first fashion, so that for any two potential merges, pair closer to the initial state is chosen. EDSM instead computes a score for all possible pairs and attempts to merge the one with the highest score.[3]

In order to determine state merging algorithm will perform best, we conduct a limited grid-

[1] (TODO: add a link to the exact version of FlexFringe we used, since it wasn't the most up-to-date. SP has this from Adil)

[2] We exclude ALERGIA from consideration because of the infeasibly high compute demands noted by Soubki and Heinz

[3] Notably, the implementation of RPNI in FlexFringe differs slightly from that in Oncina et al. (1992): while Oncina et al. merged states in length-lexicographic order when ties arise, FlexFringe instead calculates a score, akin to EDSM.

search over the following parameters provided by FLEXFRINGE:

- PERFORMFIRST: whether the algorithm should perform EDSM or RPNI
- EXTEND: extend (color red) blue states that cannot be merged with any red
- SHALLOWFIRST: consider coloring blue states closest to the root first
- SEARCHSTRATEGY: searchdeep, searchlocal, searchglobal, searchpartial, none (TODO: They have absolutely no documentation of what these parameters do, should we just guess?)

We attempted to add the following parameters to the grid search, but modifying these parameters from their default state resulted in segmentation faults in the FLEXFRINGE executable, so we left them at their default values: REVERSETRACES, DEPTHCHECK, SYMBOLCHECK, and SINKSON, SINKCOUNT, and MERGESCORE.

Grid searching was carried out on the PLUSSHORT data using overall accuracy on the development set (see §3). The best performing model, used in the remainder of the paper, was PERFORMFIRST = 0 (i.e., RPNI), EXTEND = 1, SHALLOWFIRST = 0, and SEARCHSTRATEGY = SEARCHDEEP.

2.2 Neural Systems

Following van der Poel et al. (2024), we test four neural architectures, all consisting of a trainable embedding layer followed by a unidirectional recurrent module (either a **SIMPLE** RNN, a **GRU**, or an **LSTM**) or two consecutive **TRANSFORMER** blocks, dense feed-forward layers, dropout, layer normalization, and softmax activation. All neural networks are implemented with TensorFlow and Keras (Abadi et al., 2015; Chollet, 2015) and use the same hyperparameters as van der Poel et al. (2024).[4]

3 Evaluation

We evaluate our learning models with an extension of MLREGTEST (van der Poel et al., 2024), which contains training, development, and test data with

[4]Implementations of the neural networks are available at: https://github.com/heinz-jeffrey/subregular-learning/

FACTOR	POSSIBLE LEVELS
AlphabetSize	4, 16
Tier	3, 4, 7, 16
LanguageClass	SL, coSL, TSL, TcoSL, SP, coSP, LT, TLT, PT, LTT, TLTT, PLT, TPLT, SF, Zp, Reg
FactorWidth	0, 2, 3, 4, 5, 6
ThresholdSize	0, 1, 2, 3, 5
Index	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
TrainSize	Small (1k), Mid (10k), Large (100k)
DataType	ONLYSHORT, PLUSSHORT ONLYLONG
Model	FLEXFRINGE, SIMPLE GRU, LSTM, TRANSFORMER

Table 1: The values used in our experimental setup. (SP: could put this in the appendix if we need to)

and equal number of positive and negative examples of strings from 1,800 languages. We follow Soubki and Heinz (2023) in excluding those languages with alphabet sizes of 64 due to unicode errors, thus considering 1,140 languages.

In the original version of MLREGTEST, the length of strings in training ranges from 20-29; Soubki and Heinz conducted a follow-up experiment in which roughly 1,000 short training strings (length 1-19) were added to the original MLREGTEST training data. The authors found that FLEXFRINGE significantly improved when these short strings were added to its training data. In the current work, we build on Soubki and Heinz by considering three training setups:

ONLYSHORT: models are trained on only the short strings from Soubki and Heinz (2023)

PLUSSHORT: models are trained on MLREGTEST augmented with short strings, replicating Soubki and Heinz (2023)

ONLYLONG: models are trained on the original MLREGTEST data

MLREGTEST is designed to investigate a number of potential factors which may contribute to model performance, summarized with our additions in Table 1. It additionally contains four testing sets per language, based on the length of the strings (SHORT = 20-29, LONG = 31-50) and the method of generation (RANDOM, sampled randomly without replacement or ADVERSARIAL, with paired positive and negative examples with an edit distance of 1). We report results separately on each test set.

4 Results

5 Discussion

6 Conclusions

References

Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, and 1 others. 2015. Tensorflow: Large-scale machine learning on heterogeneous systems.

François Chollet. 2015. Keras. https://github.com/fchollet/keras.

Kevin J Lang, Barak A Pearlmutter, and Rodney A Price. 1998. Results of the abbadingo one dfa learning competition and a new evidence-driven state merging algorithm. In *International Colloquium on Grammatical Inference*, pages 1–12. Springer.

José Oncina, Pedro Garcia, and 1 others. 1992. Inferring regular languages in polynomial update time. *Pattern recognition and image analysis*, 1(49-61):10–1142.

Adil Soubki and Jeffrey Heinz. 2023. Benchmarking state-merging algorithms for learning regular languages. In *Proceedings of 16th edition of the International Conference on Grammatical Inference*, volume 217 of *Proceedings of Machine Learning Research*, pages 181–198. PMLR.

Sam van der Poel, Dakotah Lambert, Kalina Kostyszyn, Tiantian Gao, Rahul Verma, Derek Andersen, Joanne Chau, Emily Peterson, Cody St. Clair, Paul Fodor, Chihiro Shibata, and Jeffrey Heinz. 2024. Mlregtest: A benchmark for the machine learning of regular languages. *Journal of Machine Learning Research*, 25(283):1–45.

Sicco Verwer and Christian Hammerschmidt. 2022. FlexFringe: Modeling software behavior by learning probabilistic automata. *arXiv preprint*.

Sicco Verwer and Christian A. Hammerschmidt. 2017. FlexFringe: A passive automaton learning package. In *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 638–642.

A Example Appendix

This is an appendix.